COO Task Management Tool

Revised specification (v2): original spec plus architecture review, Trello integration details, tool list and project skeleton

Items marked **[NEW]** are additions to the original document; items marked **[CHANGED]** replace something in it. Everything unmarked is carried over from the original.

1. Description

The agent automates team task management. The worker interacts with a bot as his/her assistant. Through audio or text input, the agent sets a new task, monitors it and manages it.

Features: create a new task; edit it (delete, set as done, move date, modify); monitor deadlines; notify the worker.

[NEW] Design principles. Only use an LLM where language understanding is needed; everything else is deterministic code. Trello is the single source of truth for tasks. No write to Trello without a confirmation from the user. Every event is processed exactly once.

2. Architecture review and revised agent list

The original flow (input, transcription, task definition, Trello, notification) is the right shape. The main change is to separate components that need an LLM from plain services, and to remove fragile hops.

2.1 What changed and why

Original component	Revised	Type	Reason
Director Agent	**[CHANGED]** Orchestrator	LLM	Decides intent (create / edit / complete / move / query) and calls Trello search tools to resolve which card is meant.
Subagent 1: Notifier (SimpleX)	SimpleX Gateway	Code	Pure I/O: receives user messages, sends notifications. No reasoning needed.
Subagent 2: Transcriptor	Transcriber	Code	Whisper runs only if the input is audio. Transcript is passed directly to the next step.
Subagent 3: Task definer	**[CHANGED]** Merged into Orchestrator	LLM	Extraction and intent are one reasoning step. Output is a structured Task object.
Subagent 4: Dispatcher	**[CHANGED]** Queue / event bus	Code	Routing does not need an LLM. A SQLite or Redis queue gives retries and idempotency.
Subagent 5: Trello Manager	Trello Service	Code	API wrapper plus webhook receiver. Emits notification events.
(missing)	**[NEW]** Scheduler	Code	Trello does not push deadline-approaching events, so reminders must be generated by us.
(missing)	**[NEW]** Confirmation loop	Code	Asks the user to confirm before any write. Protects against noisy voice transcripts.

2.2 Changes to the flow

- **[CHANGED] kDrive is no longer a trigger.** A new file in kDrive triggering the next step is fragile (sync delay, duplicate events). The transcript goes straight to the Orchestrator and is archived to kDrive afterwards.
- **[NEW] The Orchestrator reads existing state.** "Move the supplier call to Friday" only works if the agent can search open Trello cards and resolve the right one. It needs read tools, not just a form to fill.

- **[NEW] Confirmation step.** Before writing, the bot replies e.g. "Create task 'Call supplier' for Marco, due Fri 23 Oct. OK?". Mandatory for delete and for any ambiguous match.
- **[NEW] Archive instead of delete.** Deletion is implemented as archiving the card (recoverable).

2.3 Revised structure

```
SimpleX Gateway (in/out)
   -> Transcriber (Whisper, only if audio)
   -> Orchestrator LLM (intent + extraction, with Trello read tools)
   -> Confirmation loop with user (pending actions in DB)
   -> Trello Service (API wrapper, writes)

Trello webhook receiver --+
Scheduler (deadlines) ----+--> Notification builder --> SimpleX Gateway

Shared: Queue/DB (user mapping, processed IDs, pending confirmations)
```

3. List of event triggers

Trigger	Original meaning	Implemented as
1	New recording	SimpleX gateway receives an audio or text message
2	New transcription in folder	**[CHANGED]** Transcriber hands the text to the Orchestrator in code; kDrive file is an archive only
3	Identification of tasks complete	Orchestrator returns a valid Task object, then confirmation loop, then Trello write
4	In events on Trello (commands from outside)	Trello webhook: card created, updated, moved, completed, commented, member added by humans
5	Out events on Trello (reminder notification)	**[CHANGED]** Scheduler job. Trello does not send an approaching-deadline event to our endpoint

4. Task object (mirrors a Trello card)

[NEW] Draft schema returned by the Orchestrator as structured JSON.

Field	Type	Trello card field	Notes
action	enum	-	create, update, complete, move_date, archive, query
target_card_id	string or null	id	Null for create. Resolved by search for other actions
title	string	name	Short imperative, e.g. "Call supplier"
description	string	desc	Context from the original message
assignees	list of member IDs	idMembers	Resolved via the user mapping table
due	ISO 8601 datetime	due	Relative dates resolved in Europe/Rome
done	bool	dueComplete	Set by action complete
list	string	idList	e.g. To do, Doing, Done
labels	list	idLabels	Optional
confidence	0-1	-	Below a threshold the bot asks a clarifying question
source_message_id	string	-	For idempotency and audit

5. Trello integration: can everything go through the API?

[NEW] Yes. All the operations in the feature list are available through Trello's REST API. These details come from general knowledge of the API: verify endpoints and limits against the official Trello developer documentation (developer.atlassian.com/cloud/trello) before building.

5.1 Authentication

Create an API key (Power-Up admin) and generate a token with read/write scope. Use a dedicated "Task Bot" Trello account so bot actions can be told apart from human ones.

5.2 Editing tasks

Operation	Endpoint	Key parameters
Create	POST /1/cards	idList, name, desc, due, idMembers, idLabels
Modify / move date / set done / move list	PUT /1/cards/{id}	name, desc, due, dueComplete, idList, idMembers
Archive (recommended)	PUT /1/cards/{id}	closed=true (recoverable)
Permanent delete	DELETE /1/cards/{id}	Irreversible, avoid
Read / search	GET /1/boards/{id}/cards, GET /1/search	Used by the Orchestrator to resolve cards
Comments, checklists, members	Dedicated endpoints under /1/cards and /1/boards	Optional extras

5.3 Incoming events (Trello to the system): webhooks

- Register with `POST /1/webhooks` giving `idModel` (board, list or card) and a public HTTPS `callbackURL`.
- Trello posts a JSON action for each change: card created, updated (due date, list move, completion), commented, member added.
- The endpoint must answer Trello's initial HEAD request and should verify the HMAC-SHA1 signature header on every call.
- Webhooks can be disabled after repeated delivery failures: add a health check that re-registers them.
- Webhooks also fire for the bot's own edits. Ignore events whose actor is the bot's member ID to avoid loops.
- Human edits made directly in Trello arrive through the same webhook and become notifications to the people involved.

5.4 Limits to design around

- **Reminders.** There is no webhook event for an approaching deadline. Trello's own due reminders go to Trello's channels (app, email), not to our endpoint. The Scheduler fetches cards with due dates and sends reminders itself through SimpleX.
- **Outgoing messages.** Trello has no chat API. All user notifications go through SimpleX, built by our Notification builder.
- **Rate limits.** Roughly 300 requests per 10 s per API key and 100 per 10 s per token. Ample for a team, but prefer webhooks over polling.

6. Tech list

Original list: SimpleX, Whisper engine, Trello API. **[NEW]** Completed below.

Layer	Options
Chat interface	SimpleX Chat CLI/bot in WebSocket API mode, with a Python or TypeScript client
Audio	ffmpeg for format conversion; Whisper via faster-whisper or whisper.cpp (local, better for privacy) or a hosted API
LLM	Claude API with tool use and structured output (JSON schema for the Task object)
Orchestration	Plain Python/FastAPI, or the Claude Agent SDK. A heavy multi-agent framework is not needed
Webhook endpoint	FastAPI/Node server behind public HTTPS (Caddy, Cloudflare Tunnel or a small VPS)
Queue / state	SQLite or Postgres (user mapping, pending confirmations, processed message IDs); Redis optional
Scheduler	APScheduler or cron, for deadline reminders
Storage	Infomaniak kDrive via WebDAV (or the Infomaniak API) for the transcript archive, or local disk. Define the directory and file type (e.g. one .txt per message, named by date and message ID)
Deployment	Docker Compose on a VPS (Infomaniak offers hosting), secrets in .env or a secrets manager, structured logging
Task backend	Trello REST API

7. Requirements missing from the original spec

[NEW] These need a decision before or during the build.

- **User mapping table:** SimpleX contact, Trello member, role.
- **Date handling:** relative dates ("next Tuesday") resolved in Europe/Rome.
- **Languages:** mixed Italian and English input, for Whisper and for the LLM prompt.
- **Idempotency:** the same message must never create two cards (store processed message IDs).
- **Error handling and retries:** failed transcription, Trello downtime, LLM errors, with a clear message to the user.
- **Permissions:** who may assign tasks to whom, who may archive or delete.
- **Privacy / GDPR:** retention of audio files and transcripts, where they are stored, who can read them.
- **Ambiguity policy:** what the bot does when several cards match, or no assignee is named.
- **Trello board design:** lists, labels and who owns which board, so the Task object maps cleanly.

8. Project skeleton

[NEW] Suggested repository layout (Python). One module per component from section 2.

```
coo-assistant/
  README.md
  pyproject.toml
  docker-compose.yml        # app, simplex-chat, caddy (optional redis)
  .env.example              # TRELLO_KEY, TRELLO_TOKEN, ANTHROPIC_API_KEY, ...
  app/
    main.py                 # FastAPI app: webhook route, startup, scheduler
    config.py               # settings from environment
    gateway/
      simplex.py            # WebSocket client: receive msgs, send msgs
    transcribe/
      whisper_service.py    # ffmpeg + faster-whisper, language auto-detect (it/en)
    orchestrator/
      agent.py              # Claude call with tools and structured output
      prompts.py            # system prompt (roles, dates, languages, rules)
      tools.py              # search_cards, get_card, list_members
      schema.py             # Task model (section 4)
    confirm/
      pending.py            # store, approve, expire pending actions
    trello/
      client.py             # create/update/archive/search wrappers
      webhook.py            # HEAD handler, signature check, event parser
      bootstrap.py          # register webhook, health check
    notify/
      builder.py            # event -> message text per user
    scheduler/
      reminders.py          # periodic due-date scan
    storage/
      db.py                 # SQLite tables: users, processed, pending
      archive.py            # transcript archive to kDrive (WebDAV) or disk
  tests/
    test_schema.py  test_dates.py  test_webhook_signature.py  test_idempotency.py
```

8.1 Core data flow in code (outline)

```
async def handle_incoming(msg):
    if db.already_processed(msg.id):          # idempotency
        return
    text = msg.text or await transcribe(msg.audio)
    user = users.by_simplex_id(msg.sender)    # permissions + mapping
    task = await orchestrator.interpret(text, user)   # LLM + Trello read tools
    if task.needs_clarification:
        return await gateway.send(user, task.question)
    pending.save(user, task)
    await gateway.send(user, render_confirmation(task))
    db.mark_processed(msg.id)

async def on_user_confirms(user, pending_id):
    task = pending.pop(pending_id)
    result = trello.apply(task)               # create/update/archive
    await archive.save_transcript(task)
    await gateway.send(user, render_result(result))
```

8.2 Suggested build order

- Trello service and Task schema, tested against a sandbox board.
- Orchestrator with text input only, including card search and confirmation loop.
- SimpleX gateway, so the bot works end to end with text.
- Whisper transcription for audio, with Italian and English tests.
- Webhook receiver, loop prevention and notifications for human edits.
- Scheduler for deadline reminders.
- kDrive archive, GDPR retention rules, deployment and monitoring.